# Rule-Constrained Transformer Generation of Bach-Style Four-Part Chorales

**Alan Olegovich Khaev** **Saurabh Nair** **Kamil Chaudhry**

**Haris Faisal**
University of Toronto Mississauga

## Abstract

We propose a generative sequence model that composes four-part chorales in the style of Johann Sebastian Bach. Given a short musical prefix, represented as a sequence of discrete pitch tokens on a fixed time grid, the model predicts a probability distribution over the next token and generates the continuation autoregressively. Our primary model is a decoder-only Transformer trained with teacher forcing on the Bach chorales in the music21 corpus (approximately 370 pieces; TODO: confirm count). Because this dataset is small by deep learning standards, we apply key transposition as a domain-appropriate data augmentation, which increases the effective training set roughly twelvefold. Inspired by rule-based systems such as Sid Meier's CPU Bach, we additionally apply constrained decoding at generation time: candidate notes that violate basic rules of counterpoint, such as voice ranges and parallel fifths or octaves, are masked before sampling. We compare the Transformer against a first-order Markov chain and an LSTM baseline using held-out negative log-likelihood and the rate of counterpoint rule violations in generated music. This work concerns symbolic music generation, sequence modelling, and constrained decoding.

## 1 Introduction

Counterpoint is the practice of combining several independent melodic lines, called *voices*, so that they sound coherent together. The four-part chorales of J. S. Bach (soprano, alto, tenor, and bass) are a standard benchmark for computational music generation because they follow well-documented stylistic rules while still showing substantial variety. Generating convincing counterpoint is difficult: each voice must form a smooth melody on its own, while every simultaneous combination of notes must also satisfy harmonic constraints.

Two broad approaches exist. Rule-based systems encode music theory directly and produce well-formed but often predictable music. Learned models capture stylistic patterns from data but can produce errors that a trained musician would immediately reject. This project combines the two: a Transformer learns the style from data, and a small set of hard-coded rules removes clear violations during generation.

Concretely, the input to our model is a variable-length sequence of discrete tokens encoding the first few measures of a chorale, and the output is a continuation of that sequence in the same token format, which is converted back into a playable MIDI file.

A deep learning approach is reasonable here for three reasons. First, the task is naturally a sequence prediction problem in which each note depends on a long musical context, which is the setting Transformers are designed for. Second, prior neural models have produced chorales that listeners struggle to distinguish from Bach's own Hadjeres et al. [2017], Liang [2016]. Third, the rule-violation met-

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

ric gives an interpretable measure of whether the model has learned musical structure, beyond what likelihood alone reveals.

## 2 Background and Related Work

**Musical background.** A *MIDI* file stores music as a stream of events, each specifying a pitch (an integer from 0 to 127), a start time, and a duration. *Conjunct motion* refers to a voice moving by small steps, and *disjunct motion* to larger leaps; Baroque melodies favour conjunct motion. *Parallel fifths and octaves* occur when two voices move in the same direction while staying a fifth or an octave apart; this is a standard rule violation in Bach's style because it makes the voices sound less independent. A *Markov chain* predicts the next state using only the current state, with transition probabilities estimated from counts.

**Rule-based generation.** Sid Meier's CPU Bach (1994) generated Baroque-style polyphony in real time without machine learning. Its approach, described in a US patent Meier and Briggs [1996], combined Markov-chain note selection, which favours stepwise motion, with hard-coded structural rules for harmonic progression, modulation, and voice entry. We use this design in two ways: a Markov chain serves as our simplest baseline, and its rule-based component motivates our constrained decoding.

**Neural chorale generation.** Liang [2016] trained an LSTM Hochreiter and Schmidhuber [1997] on Bach chorales and reported that listeners often could not distinguish its output from the originals. Hadjeres et al. [2017] used recurrent networks with Gibbs sampling and allowed user-specified constraints, such as fixed notes, during generation. Huang et al. [2017] framed counterpoint as filling in missing notes with a convolutional network, an approach that later powered the 2019 Google Bach Doodle Huang et al. [2019a].

**Transformers for music.** The Transformer Vaswani et al. [2017] replaces recurrence with self-attention, allowing each position to attend directly to every earlier position. Huang et al. [2019b] showed that Transformers capture long-range structure in symbolic music better than LSTMs, which motivates our choice of primary model.

**Our contribution.** Prior neural chorale models learn musical rules only implicitly from data, while rule-based systems such as CPU Bach use no learning. Our project combines a Transformer with explicit counterpoint rules applied at decoding time. Unlike the constraints in DeepBach, which fix specific notes chosen by a user, our constraints are general music-theory rules, and we measure directly how often each model violates them with and without the constraints.

## 3 Data

**Source.** We use the Bach chorales included in the music21 Python library Cuthbert and Ariza [2010]. Bach's compositions are in the public domain, and the music21 corpus is distributed for research and educational use, which avoids the licensing uncertainty of individually sequenced MIDI files from other websites (TODO: confirm music21 corpus license terms). As a possible extension, we may also use the two-part Inventions and three-part Sinfonias.

**Summary statistics.** Table 1 reports the key properties of the dataset, and Figure 1 shows the distribution of chorale lengths and of pitches in each voice. (TODO: fill in using a short music21 script, and add one or two sentences on what these statistics imply for the difficulty of the task, e.g. sequence length relative to model context size.)

**Representation.** Each chorale is quantized to a grid of sixteenth-note time steps. At each step we record one token per voice in a fixed order (soprano, alto, tenor, bass). Each token is either a MIDI pitch, a *hold* token indicating that the previous note continues, or a *rest* token. The four voices are interleaved into a single sequence, so that a piece of $T$ time steps becomes a sequence of $4T$ tokens. The vocabulary therefore contains the pitches observed in the data plus the hold, rest, start, and end tokens (TODO: report final vocabulary size). This ordering means that when the model generates a

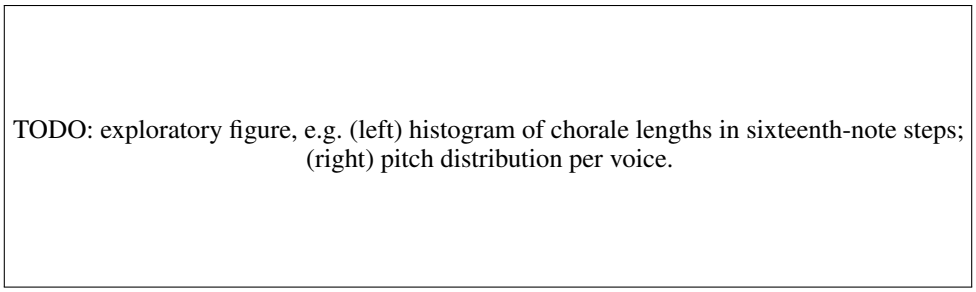


Figure 1: Exploratory statistics of the Bach chorale dataset. Left: distribution of chorale lengths in sixteenth-note time steps. Right: distribution of MIDI pitches for the soprano, alto, tenor, and bass voices. (TODO: update caption to describe the final figure.)

Table 1: Summary statistics of the Bach chorale dataset (placeholder values to be filled in).

| Statistic | Value |
|---|---|
| Number of chorales | TODO |
| Total notes (all voices) | TODO |
| Mean / max length (sixteenth-note steps) | TODO / TODO |
| Pitch range: soprano, alto, tenor, bass | TODO |
| Major / minor key proportion | TODO |
| Mean sequence length after tokenization | TODO |

bass note, the soprano, alto, and tenor notes at the same time step are already known, which allows the constraint checks described in Section 4.

**Cleaning.** We will discard chorales that do not contain exactly four voices and chorales whose note durations cannot be represented on the sixteenth-note grid, and we will report how many pieces each filter removes. (TODO: report counts.)

**Augmentation.** Because roughly 370 pieces is small for a Transformer, we transpose each training chorale to every key in which all voices remain within their observed ranges. Transposition preserves musical structure exactly, so it is an appropriate augmentation for this domain.

**Split.** We split by chorale into 80% training, 10% validation, and 10% test sets, so that test pieces are never seen in any form during training. Transposition is applied only after splitting, to prevent transposed copies of test pieces from leaking into the training set.

## 4 Model Architecture

**Primary model.** Figure 2 illustrates the overall system. Our main model is a decoder-only Transformer. Each input token is mapped to a learned embedding, to which we add a learned positional embedding and a learned voice embedding indicating which of the four voices the token belongs to. The sequence then passes through a stack of masked self-attention layers, each followed by a position-wise feed-forward network, with residual connections and layer normalization. A causal mask ensures that each position attends only to earlier positions. A final linear layer and softmax produce a distribution over the vocabulary for the next token. We train with teacher forcing, minimizing the cross-entropy between predicted distributions and the true next tokens. As a starting configuration we plan to use 6 layers, a model dimension of 256, and 4 attention heads (approximately 5 million parameters), and we will tune these values on the validation set. All models will be implemented in PyTorch Paszke et al. [2019], with music21 used for parsing and MIDI export.

**Constrained decoding.** During generation, before sampling each token, we set the probability of every candidate token that violates a rule to zero and renormalize the remaining distribution. The initial rule set is (1) each voice must stay within its typical range, (2) no parallel fifths or octaves

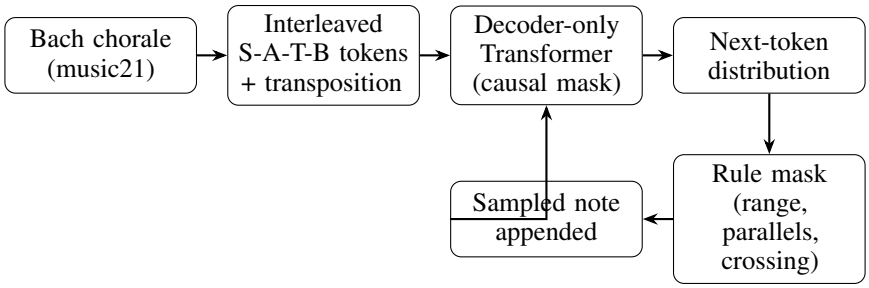


Figure 2: Overview of the proposed system. Chorales are converted into a single token sequence that interleaves the soprano, alto, tenor, and bass voices. A decoder-only Transformer predicts a distribution over the next token, a rule mask removes tokens that would break counterpoint rules, and the sampled token is appended to the input for the next step.

between any pair of voices, and (3) no voice crossing, meaning a lower voice may not sound above a higher one. These rules are applied only at generation time, so the trained model is unchanged and we can measure the effect of the constraints directly.

**Baselines.** We compare against (1) a first-order Markov chain over the same token vocabulary, which mirrors CPU Bach's note-level method, and (2) a single-layer LSTM of comparable parameter count trained with the same objective. If the Transformer does not outperform the Markov chain on held-out likelihood, we will treat that as evidence that the model has failed to learn useful structure.

**Evaluation.** We will report (1) per-token negative log-likelihood on the test set, (2) the percentage of generated time steps that violate each rule, with and without constrained decoding, and (3) the distance between pitch-class distributions of generated and real chorales. We will also present selected generated excerpts for qualitative analysis.

## 5 Ethical Considerations

**Data provenance.** Although Bach's compositions are in the public domain, a specific digital transcription can be the copyrighted work of the person who produced it. We therefore use a corpus distributed for research use rather than collecting MIDI files from arbitrary websites.

**Transfer to living artists.** The pipeline we build is independent of the composer. Applied to the work of living musicians without consent, the same method would raise the copyright and compensation concerns at the centre of current disputes over commercial music generators.

**Misattribution.** The attribution of Baroque works is not fully settled: several pieces once credited to Bach have been reassigned to other composers. Generated pieces presented as rediscovered or newly attributed works could mislead listeners and scholars, so all output from this project will be clearly labelled as machine-generated.

**Economic impact.** Systems that generate stylistically correct music cheaply compete most directly with composers of stock, library, and educational music, who are often less established than the artists usually discussed in this debate.

**Cultural bias.** Our rules and data encode one Western tradition's conventions as correct. A model or metric that treats these conventions as universal would misjudge music from other traditions, which matters if similar tools are used in education or assessment.

## 6 Work Division

Work is divided by stage of the pipeline, so that each member owns one component and the handoffs between components are clearly defined. Table 2 lists each member's responsibilities and estimated

Table 2: Division of work and estimated hours per member in each of the four project weeks.

| Member | Responsibilities | Wk 1 | Wk 2 | Wk 3 | Wk 4 |
|---|---|---|---|---|---|
| Alan | Token representation design; rule checker and constrained decoding; music-theory background; qualitative analysis of generated samples. | 6 | 5 | 5 | 4 |
| Saurabh | Transformer implementation and training loop; overfitting a single chorale; hyperparameter tuning; training curves; parameter count and architecture figure. | 3 | 7 | 7 | 3 |
| Kamil | Data loading, cleaning, and tokenization; summary statistics; transposition augmentation; data split; README and code documentation. | 8 | 4 | 3 | 5 |
| Haris | Markov chain and LSTM baselines; evaluation scripts (likelihood, rule-violation rate, pitch-class distance); results tables; final report assembly. | 3 | 6 | 6 | 5 |

weekly time commitment. Each member also writes the report section describing their component, and a second member edits it. (TODO: adjust names and hours.)

**Dependencies.** Alan and Member 3 agree on the token format jointly in the first two days, since every later component depends on it. Member 3 delivers a preliminary tokenized dataset by the end of week 1, so that the Transformer and baselines can begin training before cleaning is finalized. Saurabh and Member 4 share a single training and evaluation interface, so that all models are compared under identical conditions. If a component falls behind, the member responsible for the downstream component assists through pair programming.

**Collaboration.** The team meets weekly (TODO: day and time, e.g. Tuesdays 4–5 pm) to review progress against the timeline below, and coordinates between meetings through a group chat and a shared GitHub repository, with changes merged through pull requests reviewed by one other member.

**Timeline.** **Week 1:** data pipeline and statistics complete; Markov baseline trained; Transformer overfits a single chorale. **Week 2:** full Transformer and LSTM training; rule checker implemented. **Week 3:** hyperparameter tuning; constrained decoding integrated; evaluation run. **Week 4:** analysis, figures, README, and report writing. (TODO: align with actual course deadlines.)

## References


Michael Scott Cuthbert and Christopher Ariza. music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)*, 2010.

Gaëtan Hadjeres, François Pachet, and Frank Nielsen. DeepBach: A steerable model for Bach chorales generation. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Cheng-Zhi Anna Huang, Tim Cooijmans, Adam Roberts, Aaron Courville, and Douglas Eck. Counterpoint by convolution. In *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017.

Cheng-Zhi Anna Huang, Curtis Hawthorne, Adam Roberts, Monica Dinculescu, James Wexler, Leon Hong, and Jacob Howcroft. The Bach Doodle: Approachable music composition with machine learning at scale. In *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, 2019a.

Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music Transformer: Generating music with long-term structure. In *International Conference on Learning Representations (ICLR)*, 2019b.

Feynman Liang. BachBot: Automatic composition in the style of Bach chorales. Mphil thesis, University of Cambridge, 2016.

Sidney K. Meier and Jeffery L. Briggs. System for real-time music composition and synthesis. U.S. Patent 5,496,962, 1996.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.